

IF25-22017 PENGEMBANGAN APLIKASI MOBILE

Pertemuan 6

Networking REST API dengan Ktor

Mengambil laporan dari server, menampilkan Memuat, dan memberi pesan yang jelas saat jaringan gagal

Muhammad Habib Algifari, S.Kom., M.T.I.

Tahun 2026



Program Studi Teknik Informatika
Fakultas Teknologi Industri
Institut Teknologi Sumatera

Tiga jawaban dari tiket keluar minggu lalu

Pertanyaan	Jawaban singkat
Apa isi back stack setelah Daftar, Detail 7, lalu Form?	Daftar, Detail 7, Form. Kembali mengambil Form
Kenapa yang dikirim ke layar detail adalah id?	Kecil, selalu segar, dan bisa dipulihkan
Apa akibatnya kalau pindah tab tanpa popUpTo?	Tumpukan menumpuk, sehingga Kembali harus ditekan berkali-kali

Laporan yang tidak pernah sampai ke petugas

Lima layar kita sudah tersambung. Tetapi semua laporan masih buatan sendiri di memori. Petugas di gedung lain tidak pernah melihat laporan kalian, dan begitu aplikasi ditutup, laporan baru ikut hilang.

DISKUSI 2 MENIT

Bagaimana mengambil laporan dari server, menampilkan Memuat selama menunggu, dan memberi pesan yang jelas saat internet kampus mati?

LaporKampus

AC mati

GKU 1 R201

contoh

Proyektor rusak

GKU 2 R305

contoh

Lampu padam

Labtek 1

contoh

Semua dari SumberLaporanPalsu.
Tutup aplikasi, laporan baru hilang.

Saat pulang nanti, kalian bisa

1 **Membaca HTTP dan JSON**
Menjelaskan endpoint, method, kode status, dan JSON yang dikirim server.

2 **Memanggil server**
Mengambil data dengan Ktor dan kotlinx.serialization lewat repository.

3 **Menampilkan keadaan**
Memuat, berhasil, dan gagal dengan tombol coba lagi.

Capaian ini mendukung **CPMK0501**: menerapkan konsep pemrograman untuk pengembangan perangkat lunak, dalam bentuk aplikasi yang mengambil data dari REST API.

Bagian 1

HTTP, REST, dan JSON

Bahasa yang dipakai aplikasi dan server untuk saling meminta dan menjawab.

Satu permintaan, satu jawaban

Permintaan dari aplikasi

```
GET /laporan HTTP/1.1  
Host: alamat server kelompok  
Accept: application/json
```

Jawaban dari server

```
HTTP/1.1 200 OK  
Content-Type: application/json  
  
[ { "id": 7, "judul": "AC mati" } ]
```

Bagian	Isinya	Di contoh ini
Method	Apa yang ingin dilakukan	GET, mengambil data
Endpoint	Alamat benda yang diminta	/laporan
Kode status	Hasilnya secara singkat	200 OK, berhasil
Body	Isi data, biasanya JSON	Daftar laporan

REST: alamat untuk benda, method untuk aksi

Method	Endpoint	Artinya di LaporKampus
GET	/laporan	Ambil semua laporan
GET	/laporan/7	Ambil satu laporan dengan id 7
POST	/laporan	Kirim laporan baru, server memberi id
PUT	/laporan/7	Ganti isi laporan 7
DELETE	/laporan/7	Hapus laporan 7

Kode status: siapa yang salah

Kelompok	Contoh	Artinya	Yang ditampilkan
2xx berhasil	200, 201	Permintaan diterima, 201 berarti data baru dibuat	Data
4xx salah di klien	400, 401, 404	Permintaan keliru, belum masuk, atau data tidak ditemukan	Pesan yang bisa diperbaiki pengguna
5xx salah di server	500, 503	Server error atau sedang tidak tersedia	Pesan untuk mencoba lagi nanti

JSON dan data class yang cocok

Jawaban server

```
{
  "id": 7,
  "judul": "AC mati",
  "lokasi": "GKU 1 R201",
  "status": "Diproses",
  "petugas": "Pak Joko",
  "dibuatPada": "2026-10-05"
}
```

commonMain/data/LaporanDto.kt

```
@Serializable
data class LaporanDto(
    val id: Int,
    val judul: String,
    val lokasi: String,
    val status: String,
    val petugas: String? = null,
    val catatan: String? = null
)
```

@Serializable meminta plugin serialization membuat kode pembaca dan penulis JSON untuk kelas ini.

Nama properti harus sama dengan kunci JSON. Nilai bawaan = null membuat kunci itu boleh tidak dikirim.

BANGUN BARENG

Server mengirim kunci dibuat Pada

Data class kita tidak punya properti itu. Apa yang terjadi saat JSON ini dibaca dengan pengaturan bawaan? Tulis tebakanmu dulu.

Cek tebakanmu: kunci tambahan

Dari JSON ke data class

```
json(Json { ignoreUnknownKeys = true })
```

```
{
  "id": 7,
  "judul": "AC mati",
  "lokasi": "GKU 1 R201",
  "status": "Diproses",
  "petugas": "Pak Joko",
  "dibuatPada": "2026-10-05"
}
```

Jawaban server

```
LaporanDto(id=7, judul=AC mati, lokasi=GKU 1 R201, status=Diproses, ...)
```

```
@Serializable
data class LaporanDto(
    val id: Int,
    val judul: String,
    val lokasi: String,
    val status: String,
    val petugas: String? = null,
    val catatan: String? = null
)
```

commonMain/data/LaporanDto.kt

COBA LANGSUNG



Scan atau klik untuk membuka latihan 1: membaca JSON dari API latihan menjadi List of RemoteUser.

[Hands-on P6 di repo mata kuliah](#)

Sumber: Repo informatika-itera IF25-22017, folder P6 Networking REST API Hands-on. Bacaan lanjut: [Ktor](#) dan [SQLDelight](#), [kotlinlang.org](#)

Animasi berputar saat presentasi. Di PDF yang tampil keadaan akhirnya.

Bagian 2

Ktor client

Satu kode pemanggil server di commonMain, dengan mesin yang berbeda di tiap platform.

Dependency dan engine per platform

composeApp/build.gradle.kts

```
commonMain.dependencies {
    implementation(libs.ktor.client.core)
    implementation(libs.ktor.client.content.negotiation)
    implementation(libs.ktor.serialization.kotlinx.json)
}
androidMain.dependencies {
    implementation(libs.ktor.client.okhttp)
}
iosMain.dependencies {
    implementation(libs.ktor.client.darwin)
}
desktopMain.dependencies {
    implementation(libs.ktor.client.cio)
}
```

Platform	Engine
Android	OkHttp
iOS	Darwin
Desktop JVM	CIO, atau OkHttp

Kode di commonMain sama untuk semua. Engine adalah mesin yang benar-benar membuka koneksi di tiap platform.

Satu HttpClient untuk seluruh aplikasi

commonMain/data/Jaringan.kt

```
fun buatClient() = HttpClient {  
    install(ContentNegotiation) {  
        json(Json { ignoreUnknownKeys = true })  
    }  
    install(HttpTimeout) {  
        requestTimeoutMillis = 10_000  
    }  
    expectSuccess = true  
}
```

Bagian	Gunanya
ContentNegotiation	Mengubah JSON menjadi objek dan sebaliknya
HttpTimeout	Menyerah setelah 10 detik tanpa jawaban
expectSuccess	Kode status bukan 2xx dianggap error

`HttpClient { }` tanpa nama engine memakai engine yang ada di dependency platform itu.

GET dan POST dengan Ktor

commonMain/data/LaporanApi.kt

```
// Kelompok memakai API publik atau mock API sendiri.  
// Isi dengan alamat itu, tanpa garis miring di akhir.  
const val BASE_URL = "https://..."  
  
class LaporanApi(private val client: HttpClient) {  
    suspend fun semua(): List<LaporanDto> =  
        client.get("$BASE_URL/laporan").body()  
  
    suspend fun ambil(id: Int): LaporanDto =  
        client.get("$BASE_URL/laporan/$id").body()  
  
    suspend fun kirim(baru: LaporanBaruDto): LaporanDto =  
        client.post("$BASE_URL/laporan") {  
            contentType(ContentType.Application.Json)  
            setBody(baru)  
        }.body()  
}
```

`body()` membaca JSON menjadi tipe yang diminta fungsi. Tipe itu yang menentukan hasilnya.

`LaporanBaruDto` cukup berisi judul dan lokasi, karena id dibuat oleh server.

BANGUN BARENG

Server menjawab 500 dengan halaman error

Seandainya expectSuccess tidak dipasang. Apakah client.get melempar exception?
Lalu apa yang terjadi saat body() dipanggil?

Dari exception ke pesan yang dimengerti

commonMain/data/Pesan.kt

```
fun pesanUntuk(e: Throwable): String = when (e) {  
    is HttpRequestTimeoutException →  
        "Server terlalu lama menjawab. Coba lagi."  
    is ClientRequestException →  
        if (e.response.status.value == 404)  
            "Laporan tidak ditemukan."  
        else "Permintaan ditolak server."  
    is ServerResponseException →  
        "Server sedang bermasalah. Coba lagi nanti."  
    else → "Tidak bisa terhubung. Periksa internet."  
}
```

Kejadian	Exception
Lebih dari 10 detik	HttpRequestTimeoutException
Kode 4xx	ClientRequestException
Kode 5xx	ServerResponseException
Tidak ada internet	Exception dari engine

Bagian 3

Repository dan keadaan layar

Sumber data diganti, layar tetap sama, dan setiap keadaan punya tampilan sendiri.

Sumber yang sama, isi dari server

commonMain/data/Sumber.kt

```
interface Sumber<T> {  
    suspend fun semua(): List<T>    // baru  
    suspend fun ambil(id: Int): T  
    fun pantau(id: Int): Flow<T>  
    suspend fun kirim(baruk: T): T    // baru  
}
```

commonMain/data/SumberLaporanJaringan.kt

```
class SumberLaporanJaringan(  
    private val api: LaporanApi  
) : Sumber<Laporan> {  
    override suspend fun semua() =  
        api.semua().map { it.keLaporan() }  
  
    override suspend fun ambil(id: Int) =  
        api.ambil(id).keLaporan()  
  
    override fun pantau(id: Int) = flow { emit(ambil(id)) }  
  
    override suspend fun kirim(baruk: Laporan) =  
        api.kirim(LaporanBaruDto(baruk.judul, baruk.lokasi))  
            .keLaporan()  
}
```


Menerjemahkan DTO menjadi Laporan

commonMain/data/LaporanDto.kt

```
fun LaporanDto.keLaporan() = Laporan(  
    id = id,  
    judul = judul,  
    lokasi = lokasi,  
    fotoUrl = null,  
    status = when (status) {  
        "Diproses" → StatusLaporan.Diproses(petugas.orEmpty())  
        "Selesai" → StatusLaporan.Selesai(catatan.orEmpty())  
        else → StatusLaporan.Baru  
    }  
)
```

Teks dari server diubah menjadi sealed class `StatusLaporan` dari minggu 2.

Layar tidak pernah melihat `LaporanDto`. Kalau server mengganti nama kunci, cukup fungsi ini yang diubah.

Tiga keadaan sebagai sealed interface

commonMain/ui/DaftarUiState.kt

```
sealed interface DaftarUiState {  
    data object Memuat : DaftarUiState  
    data class Berhasil(  
        val laporan: List<Laporan>  
    ) : DaftarUiState  
    data class Gagal(  
        val pesan: String  
    ) : DaftarUiState  
}
```

	Tiga Boolean	Sealed interface
Memuat sekaligus gagal	Bisa terjadi	Tidak mungkin
Data saat gagal	Daftar lama tertinggal	Tidak ada daftar
when lupa satu kasus	Tidak ketahuan	Error saat compile

ViewModel yang memuat dari server

commonMain/ui/DaftarViewModel.kt

```
class DaftarViewModel(  
    private val sumber: Sumber<Laporan>  
) : ViewModel() {  
    private val _uiState =  
        MutableStateFlow<DaftarUiState>(Memuat)  
    val uiState = _uiState.asStateFlow()  
    init { muat() }  
  
    fun muat() { /* di samping */ }  
}
```

`muat()` dipanggil sekali di `init` dan dipanggil lagi oleh tombol Coba lagi. `Memuat`, `Berhasil`, dan `Gagal` diimpor dari `DaftarUiState`.

Isi fungsi `muat()`

```
fun muat() {  
    viewModelScope.launch {  
        _uiState.value = Memuat  
        _uiState.value = try {  
            Berhasil(sumber.semua())  
        } catch (e: CancellationException) {  
            throw e  
        } catch (e: Exception) {  
            Gagal(pesanUntuk(e))  
        }  
    }  
}
```


Layar memilih tampilan dengan when

commonMain/ui/DaftarScreen.kt

```
@Composable
fun DaftarScreen(
    vm: DaftarViewModel,
    bukaDetail: (Int) → Unit,
    bukaForm: () → Unit
) {
    val state by vm.uiState.collectAsState()
    when (val s = state) {
        Memuat → CircularProgressIndicator()
        is Berhasil →
            LayarDaftar(s.laporan, bukaDetail, bukaForm)
        is Gagal → PesanGagal(s.pesan, vm::muat)
    }
}
```

commonMain/ui/PesanGagal.kt

```
@Composable
fun PesanGagal(
    pesan: String,
    onCobaLagi: () → Unit
) {
    Column {
        Text(pesan)
        Button(onClick = onCobaLagi) {
            Text("Coba lagi")
        }
    }
}
```

LayarDaftar dari minggu 3 dan 5 dipakai tanpa diubah.

Memuat, berhasil, gagal, coba lagi

Permintaan, jawaban, dan keadaan layar Skenario 2: server tidak menjawab

LaporKampus

- AC mati**
GKU 1 R201
- Proyektor rusak**
GKU 2 R305
- Lampu padam**
Labtek 1

```
uiState = DaftarUiState.Berhasil(laporan)
```

permintaan

jawaban

Server
BASE_URL
/laporan

Riwayat uiState
Memuat Berhasil Memuat **Gagal** Memuat Berhasil

Kali ini server menjawab, daftar tampil lagi.

COBA LANGSUNG



Scan atau klik, tekan Run, lalu ubah ke `== 1` menjadi ke `<= 2`. Keadaan apa yang tercetak?

[Jalankan simulasinya di Kotlin Playground](#)

Sumber: Kotlin Playground, JetBrains. Kode memakai `kotlinx.coroutines` dan `sealed interface` seperti di slide.

Animasi berputar saat presentasi. Di PDF yang tampil keadaan akhirnya.

Form akhirnya mengirim ke server

commonMain/ui/FormLaporanViewModel.kt

```
fun kirim() {  
    val s = _uiState.value  
    if (!s.bisaKirim) return  
    viewModelScope.launch {  
        _uiState.value = s.copy(sedangMengirim = true)  
        _uiState.value = try {  
            sumber.kirim(s.keLaporan())  
            FormLaporanUiState(pesan = "Laporan terkirim")  
        } catch (e: CancellationException) {  
            throw e  
        } catch (e: Exception) {  
            s.copy(pesan = pesanUntuk(e))  
        }  
    }  
}
```

Baris `delay(800)` dari minggu 4 kini diganti `sumber.kirim`.

Kalau gagal, isi form tidak dikosongkan. Pengguna cukup menekan Kirim lagi tanpa mengetik ulang.

Bagian 4

Coba sendiri, lalu bawa ke proyekmu

Tiga latihan Ktor dengan observasi praktik, data REST untuk tema kelompok, dan checkpoint proyek.

Yang saya amati selama latihan hari ini

Indikator RPS: **Data dari API tampil dengan loading/error states**. Diamati dari latihan 2 dan 3, Repository dan UserListUi.

Kriteria	Sangat baik 85 sampai 100	Baik 70 sampai 84	Cukup 55 sampai 69	Kurang di bawah 55
Data API dengan keadaan memuat dan error CPMK0501, 4%	Data tampil lewat repository, keadaan memuat dan error tampil, tombol Coba Lagi jalan	Data tampil, satu keadaan belum ditangani	Data tampil tanpa keadaan memuat dan error	Belum memanggil API

Individu, bobot 4% nilai akhir. Bukti: push ke repository latihan pribadimu, lalu kirim tautan commit di LMS sebelum kelas selesai.

Tiga latihan di repo mata kuliah

Latihan 1

Handson1KtorClient.kt
HttpClient dengan
ContentNegotiation, lalu fetchUsers
yang membaca List of RemoteUser.

Latihan 2

Handson2Repository.kt
UserRepository dan load() yang
mengubah keadaan Loading,
Success, atau Error.

Latihan 3

Handson3UserListUi.kt
Indikator memuat, LazyColumn, dan
pesan error dengan tombol Coba
Lagi.

Semuanya di folder **P6 Networking REST API Hands-on**, memanggil `jsonplaceholder.typicode.com/users`. Perlu internet. Yang sudah selesai, ganti sealed class di latihan 2 menjadi sealed interface.

Kalau error

Gejala atau pesan	Penyebab	Yang dicoba
Permission denied, missing INTERNET permission	Izin belum ada di AndroidManifest.xml	Tambahkan uses-permission INTERNET
No client engine found	Engine platform itu belum ada	Engine di setiap source set
SerializationException	Nama properti tidak cocok atau ada kunci tambahan	Cocokkan nama, ignoreUnknownKeys
Layar tidak pernah keluar dari Memuat	load() tidak dipanggil, atau exception tertelan	LaunchedEffect(Unit) dan try catch lengkap

Data REST untuk tema kelompokmu

1 **Pilih servernya**
API publik yang sesuai tema, atau mock API buatan kelompok sendiri.

2 **DTO dan Api**
Data class @Serializable dan kelas Api dengan GET dan POST.

3 **Sumber jaringan**
Implementasi Sumber untuk entitas utama, layar tidak diubah.

4 **Tiga keadaan**
Memuat, berhasil, dan gagal dengan tombol coba lagi di layar daftar.

CONTOH DARI LAPORKAMPUS

LaporanDto, LaporanApi, dan SumberLaporanJaringan. DaftarUiState dengan Memuat, Berhasil, dan Gagal. BASE_URL disimpan di satu konstanta.

Checkpoint proyek minggu 6

- 1 Data dari server**
REST API publik atau mock API buatan sendiri, dipanggil dengan Ktor.
- 2 Lewat repository**
Implementasi Sumber untuk entitas utama, layar tetap memakai ViewModel.
- 3 Memuat dan error**
Tiga keadaan tampil, error punya pesan jelas dan tombol coba lagi.

Dikumpulkan

Push ke repository kelompok. README memuat alamat API yang dipakai dan tangkapan layar keadaan memuat dan error.

Dicek di awal pertemuan berikutnya: tuntas, sebagian, atau belum.

TUTUP

Tiket keluar



1 HTTP

Apa arti kode status 404 dan 500, dan mana yang kesalahan di sisi klien?

2 JSON

Kenapa data class butuh `@Serializable`, dan apa yang terjadi bila server mengirim field tambahan?

3 Keadaan

Kenapa keadaan layar ditulis sebagai sealed interface, bukan tiga Boolean?

Jawab singkat di kertas atau formulir LMS sebelum keluar kelas.

TUTUP

Minggu depan: tetap jalan saat offline



MASALAH MINGGU 7

Di gedung kuliah sinyal sering hilang. Begitu offline, daftar laporan kosong dan draf yang sedang ditulis ikut hilang. Bagaimana aplikasi tetap menampilkan data terakhir dan menyimpan draf?

Persiapan

Data REST kelompok sudah tampil dengan tiga keadaan, dan latihan Ktor sudah di-push ke repository latihan pribadi.

Terima kasih

Sampai jumpa di pertemuan 7

Ada pertanyaan? Sampaikan sekarang, lewat LMS, atau email ke muhammad.algifari@if.itera.ac.id